Q-learning

Mathematics, exploration, and off-policy learning.

Learn values

Bellman targets
Temporal-difference updates

Understand behavior

Exploration / exploitation
Off-policy learning

30 min explanation · 10 min demo · 5 min recap

From planning and imitation to RL

Keep the policy idea; learn from a different signal.

Week 6 · Planning

Known model → Bellman backups

Week 7 · Imitation

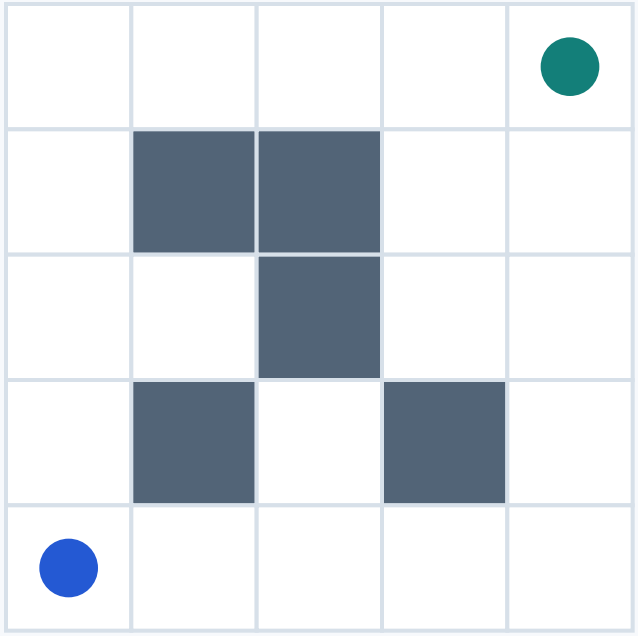
Expert observations → action labels

Week 8 · RL

Sampled transitions → reward feedback

Our maze and interaction loop

State → action → reward and next state → update.



Goal (0,4)

Start (4,0)

Agent

Select an action; update Q.

↓

Environment

Return next state and reward.

Ordinary / blocked move: −1. Goal arrival: +10, then terminate.

Reward versus return

A good action can have a bad immediate reward.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k}$$

Eight moves: $-1, -1, -1, -1, -1, -1, -1, +10$

Raw sum $= 3$ | Discounted return ≈ 0.950119

γ = 0.95 discounts future rewards; it is not the exploration rate.

What does Q(s,a) mean?

Take this action now; account for the continuation.

$$Q^{\pi}(s,a) = \mathbb{E}[G_t \mid S_t{=}s, \text{ take } a, \text{ then follow } \pi]$$

$Q^{*}(s,a)$: optimal expected continuation

$Q_t(s,a)$: current estimate of $Q^{*}(s,a)$

A Q-value is a return estimate, not an action probability.

Max returns a value; argmax an action

Choose from the four values at the current state.

$$Q(s,\cdot) = [4, 1, -2, 0]$$

$$\max_a Q(s,a) = 4 \quad | \quad \operatorname{argmax}_a Q(s,a) = \text{up}$$

$$V^*(s) = \max_a Q^*(s,a)$$

$$\pi_Q(s) \in \operatorname{argmax}_a Q(s,a)$$

Action order: up / right / down / left.

Bellman optimality for Q

Immediate reward plus the best continuation at the next state.

$$G_t = R_t + \gamma G_{t+1}$$

$$Q^*(s,a) = \mathbb{E}[\, R_t + \gamma(1-D_t)$$

$$\times \max_{a'} Q^*(S_{t+1},a') \mid S_t=s,\, A_t=a \,]$$

Average successor outcomes; maximize next-state actions.

From a model expectation to experience

Use the transition we actually observed.

Known-model backup

Average all possible outcomes using transition probabilities.

→

Sampled experience

Observe one outcome: (s, a, r, s', d)

Sample target: $r + \gamma(1-d)\max_{a'} Q(s',a')$

One sample and one visited entry; no full model sweep.

Derive the Q-learning update

Target → temporal-difference error → correction.

$$y_t = r_t + \gamma(1-d_t)\max_{a'} Q_t(s_{t+1},a')$$

$$\delta_t = y_t - Q_t(s_t,a_t)$$

$$Q_{t+1}(s_t,a_t) = Q_t(s_t,a_t) + \alpha\delta_t$$

$$= (1-\alpha)Q_t(s_t,a_t) + \alpha y_t$$

α = 0.2: move 20% toward the target. Other entries stay fixed.

Calculate one transition

Old Q = 0; reward = −1; next-state maximum = 4.

$$y = -1 + 0.95(4) = 2.8$$

$$\delta = 2.8 - 0 = 2.8$$

$$Q_{new} = 0 + 0.2(2.8) = 0.56$$

These values illustrate the update; they are not the first zero-table step.

Termination is different from truncation

Goal ends the task; the step cap ends collection.

Goal · termination

Target = reward = 10
Old = 0; new = 2
No continuation.

Cap · truncation

Target = r + γ max Q(s',·)
Keep continuation.
Collection then restarts.

At the 100-step collection cap: retain the bootstrap, then reset.

Why the maximum in the target?

Estimate the best continuation, even if behavior explores.

$$\text{At } s': \quad Q(s', \cdot) = [4, 1, -2, 0]$$

Next behavior action

Exploration may select value −2.

Continuation target

The max still uses value 4.

The max does not claim that the next behavior action will be greedy.

Q-learning is off-policy

The behavior that gathers data can differ from the target.

Behavior policy μ

Epsilon-greedy
Chooses experienced actions.

Greedy target π_Q

Max at the next state
Values greedy continuation.

Data: μ | Target: $r + \gamma \max_{a'} Q(s', a')$

Behavior explores; the target assumes greedy continuation.

Compare with SARSA

Same transition; different next-action target.

Q-learning

Uses max next Q = 4
Target = 2.8
New = 0.56

SARSA

Uses chosen next Q = −2
Target = −2.9
New = −0.58

SARSA is a comparison only; our demo implements Q-learning.

Why explore and exploit?

Learn about alternatives while using what already seems good.

Explore

Try alternatives.
Improve data coverage.
May cost extra steps.

Exploit

Use the current best estimate.
Improve immediate behavior.
Can miss unknown alternatives.

Exploration discovers information; exploitation uses current estimates.

Epsilon-greedy: actual probabilities

Random exploration includes the greedy action too.

$$G_Q(s) = \{a : Q(s,a) = \max_b Q(s,b)\}$$

$$\mu(a|s) = \varepsilon/4 + (1-\varepsilon)\,\mathbb{1}[a \in G_Q(s)] \,/\, |G_Q(s)|$$

One greedy action

0.8 + 0.05 = 0.85

Each other action

0.05

With one greedy action: 85% greedy action, 15% other actions.

Exploration in our implementation

The slider changes behavior, not the Q-learning target.

Training

ε = 0.2; slider adjustable
No automatic decay
Random greedy ties

Display / evaluation

Fixed tie order:
up → right → down → left
Evaluation stays greedy

Reset clears Q and restores the seed, retaining your selected epsilon.

Training versus evaluation

Measure the learned policy without changing the training run.

Training

Epsilon-greedy behavior
Update visited Q entry
Continue collecting experience

Evaluation

Frozen Q; greedy actions
Preserve position/history/RNG
Measure the policy

Epsilon = 0 alone does not disable learning; evaluation skips updates.

Read the algorithm

A sampled transition changes one table entry.

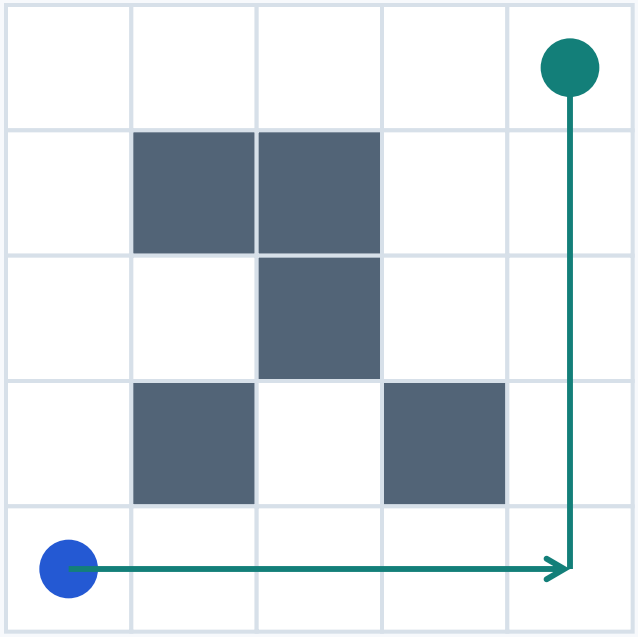
```
old = float(q[state+(action,)])
target = reward + (0 if terminated else
                   gamma*float(q[nxt].max()))
new = old + alpha*(target-old)
q[state+(action,)] = new
```

```
exploratory = bool(self.rng.random() < eps)
```

Choose → observe → update one entry → repeat / reset.

Interpret the learning results

Training behavior and greedy evaluation can have different lengths.



Training

Exploratory episode lengths vary.

Greedy evaluation

Saved seed-7 result: 8 moves.

Eight moves → raw return 3; discounted return ≈ 0.950119.

What does this result establish?

A successful finite run is not a general convergence proof.

What we observe

A learned greedy route
Under this maze and start
After finite training

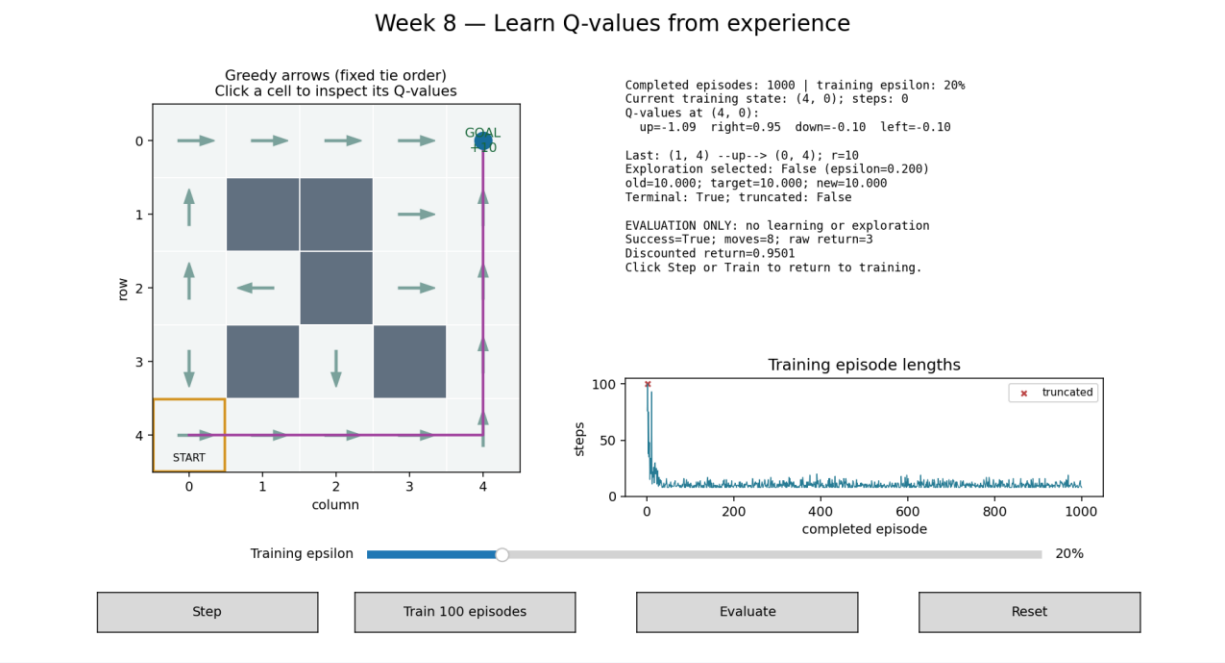
What needs more evidence

Global coverage / optimality
Robustness to changes
General convergence

Coverage and learning-rate conditions matter beyond this demonstration.

Predict the update, then train

Inspect → Step → Train → Evaluate.



Predict a Q update.

Train; inspect values.

Evaluate greedily.

3 min updates · 2 batch · 3 greedy route · 2 epsilon/reset

Questions & recap

Explain the target, the behavior, and the measurement.

Mathematics

Compute target, TD error, and new Q.

Off-policy

Separate behavior and target.

Evaluation

Freeze learning; measure the policy.

Week 9: neural policies for continuous control.

OPTIONAL / DEEPER DISCUSSION

Bellman expectation and sample targets

Optional derivation for a general stochastic MDP.

$$Q^*(s,a) = \sum_{s'} P(s'|s,a) \{ r(s,a,s')$$

$$+ \gamma[1-d(s')] \max_{a'} Q^*(s',a') \}$$

$$\mathbb{E}[y_t \mid s_t=s, a_t=a, Q_t] = (TQ_t)(s,a)$$

A sample targets the current Bellman backup, not an already known Q*.

OPTIONAL / DEEPER DISCUSSION

Convergence conditions and step sizes

Finite discounted tabular setting; sufficient repeated coverage.

For every (s,a): $\sum_n \alpha_n(s,a) = \infty$

$\sum_n \alpha_n(s,a)^2 < \infty$

Example: $\alpha_n(s,a) = 1 / N_n(s,a)$

Our constant α = 0.2 and finite training are teaching defaults.

OPTIONAL / DEEPER DISCUSSION

Q-learning versus SARSA: extended check

Same old table and nonterminal sample in both calculations.

Q-learning: $y = -1 + 0.95 \max\{4,1,-2,0\} = 2.8$

SARSA: $y = -1 + 0.95(-2) = -2.9$

Old $= 0$, $\alpha = 0.2$: Q new $= 0.56$; SARSA new $= -0.58$

If the selected next action is greedy, the two targets may agree.

OPTIONAL / DEEPER DISCUSSION

The maximum can favor noisy estimates

Optional motivation for Double Q-learning.

$$\mathbb{E}[\max_a \hat{Q}(s,a)] \geq \max_a \mathbb{E}[\hat{Q}(s,a)]$$

One estimator

Select and evaluate its maximum.

Double Q idea

Separate selection and evaluation.

Useful optimality logic does not make estimated maxima unbiased.

OPTIONAL / DEEPER DISCUSSION

Termination, truncation, and code

The bootstrap guard checks termination alone.

```
target = reward + (0 if terminated else
                   gamma*float(q[nxt].max()))
```

Goal

terminated = True

100-step cap

truncated = True; keep bootstrap

Use the observed next state before resetting collection.